TITLE : Movies, OTT & Entertainment – SQL Major Project Dataset

```
CREATE DATABASE OTT_MOVIES;

#MOVIES TABLE

CREATE TABLE movies (
   movie_id INT PRIMARY KEY,
   movie_name VARCHAR(100),
   genre VARCHAR(30),
   language VARCHAR(30),
   release_year INT,
   duration INT, -- minutes
   rating DECIMAL(3,1)
);

#Insert Data

INSERT INTO movies VALUES
(1,'Jai Bhim','Drama','Tamil',2021,164,8.8),
(2,'RRR','Action','Telugu',2022,187,8.6),
(3,'KGF Chapter 2','Action','Kannada',2022,168,8.4),
(4,'Vikram','Action','Tamil',2022,175,8.3),
(5,'Soorarai Pottru','Drama','Tamil',2020,153,8.7),
(6,'Drishyam 2','Thriller','Malayalam',2021,152,8.2),
(7,'Dangal','Sports','Hindi',2016,161,8.4),
(8,'Baahubali 2','Fantasy','Telugu',2017,171,8.2),
(9,'Andhadhun','Thriller','Hindi',2018,139,8.2),
```

```
(10,'Kantara','Drama','Kannada',2022,148,8.3),
(11,'Leo','Action','Tamil',2023,164,7.8),
(12,'Jawan','Action','Hindi',2023,169,7.9),
(13,'Pushpa','Action','Telugu',2021,179,7.6),
(14,'3 Idiots','Comedy','Hindi',2009,170,8.4),
(15,'Premam','Romance','Malayalam',2015,156,8.3);

#ACTORS TABLE

CREATE TABLE actors (
  actor_id INT PRIMARY KEY,
  actor_name VARCHAR(50),
  gender VARCHAR(10)
);

INSERT INTO actors VALUES
(1,'Surya','Male'),
(2,'Vijay','Male'),
(3,'Kamal Haasan','Male'),
(4,'Yash','Male'),
(5,'Ram Charan','Male'),
(6,'Jr NTR','Male'),
(7,'Aamir Khan','Male'),
(8,'Prithviraj','Male'),
(9,'Rajkummar Rao','Male'),
(10,'Alia Bhatt','Female');

# MOVIE_CAST TABLE (Many-to-Many)
```

```
CREATE TABLE movie_cast (
   movie_id INT,
   actor_id INT,
   role_name VARCHAR(50),
   PRIMARY KEY (movie_id, actor_id)
);

INSERT INTO movie_cast VALUES
(1,1,'Lawyer'),
(2,5,'Freedom Fighter'),
(2,6,'Revolutionary'),
(3,4,'Rocky'),
(4,3,'Agent Vikram'),
(5,1,'Pilot'),
(6,8,'Father'),
(7,7,'Wrestler'),
(8,5,'Prince'),
(9,9,'Blind Pianist'),
(10,4,'Tribal Hero'),
(11,2,'Gangster'),
(12,2,'Vigilante'),
(13,4,'Smuggler'),
(14,7,'Student'),
(15,8,'College Student');

# OTT PLATFORMS TABLE
```

```
CREATE TABLE ott_platforms (
   platform_id INT PRIMARY KEY,
   platform_name VARCHAR(30),
   launch_year INT
);

INSERT INTO ott_platforms VALUES
(1,'Netflix',2016),
(2,'Amazon Prime',2016),
(3,'Disney+ Hotstar',2019),
(4,'Zee5',2018),
(5,'Sony LIV',2020);

# MOVIE_STREAMING TABLE

CREATE TABLE movie_streaming (
   movie_id INT,
   platform_id INT,
   streaming_year INT,
   PRIMARY KEY (movie_id, platform_id)
);

INSERT INTO movie_streaming VALUES
(1,2,2021),
(2,1,2022),
(3,2,2022),
(4,3,2022),
(5,2,2020),
```

```
(6,3,2021),
(7,1,2017),
(8,1,2018),
(9,2,2019),
(10,4,2022),
(11,3,2023),
(12,1,2023),
(13,2,2021),
(14,1,2010),
(15,2,2016);

# USERS TABLE

CREATE TABLE users (
   user_id INT PRIMARY KEY,
   user_name VARCHAR(50),
   city VARCHAR(30)
);

INSERT INTO users VALUES
(1,'Arun','Chennai'),
(2,'Divya','Coimbatore'),
(3,'Rahul','Bangalore'),
(4,'Sneha','Hyderabad'),
(5,'Karthik','Mumbai');

# VIEWING HISTORY TABLE
```

```
CREATE TABLE viewing_history (
   view_id INT PRIMARY KEY,
   user_id INT,
   movie_id INT,
   watch_date DATE,
   watch_hours DECIMAL(3,1)
);

INSERT INTO viewing_history VALUES
(1,1,1,'2023-01-10',2.5),
(2,1,4,'2023-02-12',3.0),
(3,2,2,'2023-03-15',2.8),
(4,3,3,'2023-04-18',3.1),
(5,4,5,'2023-05-20',2.6),
(6,5,7,'2023-06-25',2.7),
(7,2,10,'2023-07-01',2.4),
(8,3,11,'2023-08-05',2.9);

#PROJECT LEARNING OUTCOMES

#Students can practice: - Single-table queries - Multi-table JOINs - GROUP BY,
#HAVING - Subqueries - Ranking & analytics - OTT platform analysis

# SAMPLE MAJOR PROJECT QUESTIONS

# 🔹 PART A – Single Table Queries (Movies / OTT / Users)

#1. Display all movies.
```

```
SELECT * FROM movies;
```

#2. List movie name, genre, and rating.

```
SELECT movie_name, genre, rating
FROM movies;
```

#3. Display all action movies.

```
SELECT * FROM movies
WHERE genre = 'Action';
```

#4. List movies released after 2020.

```
SELECT * FROM movies
WHERE release_year > 2020;
```

#5. Display movies with rating above 8.5.

```
SELECT * FROM movies WHERE
rating > 8.5;
```

#6. Show distinct movie languages.

```
SELECT DISTINCT language
FROM movies;
```

#7. Display top 5 highest-rated movies.

```
SELECT * FROM movies
ORDER BY rating DESC
LIMIT 5;
```

#8. Count total number of movies.

```
SELECT COUNT(*) AS total_movies
FROM movies;
```

#9. Find average movie duration.

```
SELECT AVG(duration) AS avg_duration
FROM movies;
```

#10.Display movies ordered by release year.

```
SELECT * FROM movies
ORDER BY release_year;
```

🔹 PART B – GROUP BY & Aggregates

#11. Genre-wise movie count.

```
SELECT genre, COUNT(*) AS genrewise_movie_count
FROM movies
GROUP BY genre;
```

#12. Language-wise average rating.

```
SELECT language, AVG(rating)
FROM movies
GROUP BY language;
```

#13. Find maximum rating per genre.

```
SELECT genre, MAX(rating) AS max_rating_per_genre
FROM movies
GROUP BY genre;
```

#14.Count movies released each year.

```
SELECT release_year, COUNT(*) AS rel_year_count
FROM movies
GROUP BY release_year;
```

#15.Display genres having more than 2 movies.

```
SELECT genre
FROM movies
GROUP BY genre
HAVING COUNT(*) > 2;
```

🔹 PART C – JOIN Queries (Movies, OTT, Streaming)

#16. List movies with their OTT platforms.

```
SELECT m.movie_name, o.platform_name
FROM movies m
JOIN movie_streaming ms ON m.movie_id = ms.movie_id
JOIN ott_platforms o ON ms.platform_id = o.platform_id;
```

#17.Display movies available on Netflix.

```
SELECT m.movie_name
FROM movies m
JOIN movie_streaming ms ON m.movie_id = ms.movie_id
JOIN ott_platforms o ON ms.platform_id = o.platform_id
WHERE o.platform_name = 'Netflix';
```

#18.Count movies per OTT platform.

```
SELECT o.platform_name, COUNT(ms.movie_id)
FROM ott_platforms o
JOIN movie_streaming ms ON o.platform_id = ms.platform_id
GROUP BY o.platform_name;
```

#19. Find OTT platform with maximum movies.

```
 SELECT o.platform_name, count(*) AS max_movie_count
FROM ott_platforms o
JOIN movie_streaming ms ON o.platform_id = ms.platform_id
GROUP BY o.platform_name
```

```
ORDER BY COUNT(*) DESC
LIMIT 1;
```

#20.Display movies streamed after 2021.

```
SELECT m.movie_name
FROM movies m
JOIN movie_streaming ms ON m.movie_id = ms.movie_id
WHERE ms.streaming_year > '2021-12-31';
```

🔹 PART D – Actor & Cast Analysis

#21. List actors and their movies.

```
SELECT a.actor_name, m.movie_name
FROM actors a
JOIN movie_cast mc ON a.actor_id = mc.actor_id
JOIN movies m ON mc.movie_id = m.movie_id;
```

#22.Count number of movies per actor.

```
SELECT a.actor_name, COUNT(mc.movie_id) AS actor_movies_count
FROM actors a
JOIN movie_cast mc ON a.actor_id = mc.actor_id
GROUP BY a.actor_name;
```

#23. Actor who acted in maximum movies.

```
SELECT a.actor_name, COUNT(*) AS max_mov_actor_count
FROM actors a
JOIN movie_cast mc ON a.actor_id = mc.actor_id
GROUP BY a.actor_name
ORDER BY COUNT(*) DESC
LIMIT 1;
```

#24. List movies featuring actor ‘Yash’.

```
SELECT m.movie_name
FROM movies m
JOIN movie_cast mc ON m.movie_id = mc.movie_id
JOIN actors a ON mc.actor_id = a.actor_id
WHERE a.actor_name = 'Yash';
```

#25. Find actors involved in action movies.

```
 SELECT DISTINCT a.actor_name
FROM actors a
JOIN movie_cast mc ON a.actor_id = mc.actor_id
JOIN movies m ON mc.movie_id = m.movie_id
WHERE m.genre = 'Action';
```

🔹 PART E – User Viewing Analysis

#26. List users and movies they watched.

```
SELECT u.user_name, m.movie_name
```

```
FROM users u
JOIN viewing_history v ON u.user_id = v.user_id
JOIN movies m ON v.movie_id = m.movie_id;
```

#27.Count movies watched by each user.

```
SELECT u.user_name, COUNT(v.movie_id)
FROM users u
JOIN viewing_history v ON u.user_id = v.user_id
GROUP BY u.user_name;
```

#28.User who watched maximum movies.

```
SELECT u.user_name AS max_user
FROM users u
JOIN viewing_history v ON u.user_id = v.user_id
GROUP BY u.user_name
ORDER BY COUNT(*) DESC
LIMiT 1;
```

#29.Most watched movie.

```
SELECT m.movie_name, COUNT(*) AS most_watched_movies
FROM movies m
JOIN viewing_history v ON m.movie_id = v.movie_id
GROUP BY m.movie_name
ORDER BY COUNT(*) DESC
```

```
LIMIT 1;
```

#30. Total watch hours per movie.

```
SELECT m.movie_name, SUM(v.watch_hours)
FROM movies m
JOIN viewing_history v ON m.movie_id = v.movie_id
GROUP BY m.movie_name;
```

🔹 PART F – Subqueries & Analytics

#31.Movies with rating above average.

```
SELECT * FROM movies
WHERE rating >
        (
  SELECT AVG(rating)
        FROM movies
   );
```

#32.Movies longer than average duration.

```
SELECT * FROM movies
  WHERE duration >
  (
  SELECT AVG(duration)
  FROM movies
  );
```

#33. OTT platforms hosting action movies.

```
SELECT DISTINCT o.platform_name
FROM ott_platforms o
JOIN movie_streaming ms ON o.platform_id = ms.platform_id
JOIN movies m ON ms.movie_id = m.movie_id
WHERE m.genre = 'Action';
```

#34.Users who watched movies rated above 8.5.

```
SELECT DISTINCT u.user_name
FROM users u
JOIN viewing_history v ON u.user_id = v.user_id
JOIN movies m ON v.movie_id = m.movie_id
WHERE m.rating > 8.5;
```

#35.Movies not yet watched by any user.

```
SELECT *
FROM movies
WHERE movie_id NOT IN
        (
  SELECT movie_id
  FROM viewing_history
  );
```

🔹 PART G – Date & Reporting Queries

#36.Movies watched in 2023.

```
SELECT DISTINCT m.movie_name
FROM movies m
JOIN viewing_history v ON m.movie_id = v.movie_id
WHERE YEAR(v.watch_date) = 2023;
```

#37.Users who watched movies in multiple months.

```
SELECT u.user_name
FROM users u
JOIN viewing_history v ON u.user_id = v.user_id
GROUP BY u.user_name
HAVING COUNT(DISTINCT MONTH(v.watch_date)) > 1;
```

#38.Movies streamed before release year.

```
SELECT m.movie_name
FROM movies m
JOIN movie_streaming ms ON m.movie_id = ms.movie_id
WHERE YEAR(ms.streaming_year) < m.release_year;
```

#39. Latest released movie.

```
SELECT * FROM movies ORDER BY release_year DESC LIMIT 1;
```

#40. Oldest movie in dataset.

```
SELECT * FROM movies ORDER BY release_year LIMIT 1;
```

🔹 PART H – Ranking & Advanced (Optional)

#41. Rank movies by rating.

```
SELECT movie_name, rating,
    RANK() OVER (ORDER BY rating DESC) AS rank_no
FROM movies;
```

#42. Rank actors by movie count.

```
SELECT a.actor_name, COUNT(mc.movie_id) AS movie_count,
RANK() OVER(ORDER BY COUNT(mc. movie_id) DESC) AS mov_count_by_actorsrank
FROM actors a
JOIN movie_cast mc ON a.actor_id = mc.actor_id
GROUP BY a.actor_name;
```

#43. Platform-wise average movie rating.

```
SELECT p.platform_name, AVG(m.rating) AS avg_rating
FROM ott_platforms p
JOIN movie_streaming ms ON p.platform_id = ms.platform_id
JOIN movies m ON ms.movie_id = m.movie_id
GROUP BY p.platform_name;
```

#44.Most popular genre based on views.

```
SELECT m.genre, COUNT(v.view_id) AS total_views
FROM movies m
JOIN viewing_history v ON m.movie_id = v.movie_id
GROUP BY m.genre
ORDER BY total_views DESC
LIMIT 1;
```

#45.City-wise user watch count.

```
SELECT u.city, COUNT(v.view_id) AS watch_count
FROM users u
JOIN viewing_history v ON u.user_id = v.user_id
GROUP BY u.city;
```

🔹 PART I – Case Study / Business Queries

#46.Categorize movies based on rating.

```
SELECT movie_name, rating,
    CASE
      WHEN rating >= 8.5 THEN 'Blockbuster'
      WHEN rating >= 8.0 THEN 'Super Hit'
      WHEN rating >= 7.5 THEN 'Hit'
      ELSE 'Average'
    END AS movie_category
FROM movies;
```

#47. Identify binge-watch users (watch_hours > 2.5).

```
SELECT DISTINCT u.user_name, v.watch_hours
FROM users u
JOIN viewing_history v ON u.user_id = v.user_id
WHERE v.watch_hours > 2.5;
```

#48.Movies available on more than one platform.

```
SELECT m.movie_name, COUNT(ms.platform_id) AS platform_count
FROM movies m
JOIN movie_streaming ms ON m.movie_id = ms.movie_id
GROUP BY m.movie_name
HAVING COUNT(ms.platform_id) > 1;
```

#49. Find platform with highest total watch hours.

```
SELECT p.platform_name, SUM(v.watch_hours) AS total_watch_hours
FROM ott_platforms p
JOIN movie_streaming ms ON p.platform_id = ms.platform_id
JOIN viewing_history v ON ms.movie_id = v.movie_id
GROUP BY p.platform_name
ORDER BY total_watch_hours DESC
LIMIT 1;
```

#50. Generate movie performance report.

```
SELECT m.movie_name,
    m.genre,
    m.rating,
    COUNT(v.view_id) AS total_views,
    SUM(v.watch_hours) AS total_watch_hours
FROM movies m
LEFT JOIN viewing_history v ON m.movie_id = v.movie_id
GROUP BY m.movie_name, m.genre, m.rating
ORDER BY total_watch_hours DESC;
```